

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

CS 498: Fine-Tuning Techniques

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy, dLoRA team, and many colleagues

Recall: Model Training is Expensive

- **Model parallelism: distribute model execution across machines**
  - {Data, Tensor, Pipeline} parallelism

- **Mixed-precision training may hurt quality**
  - Activation checkpointing lowers memory footprint but increases computation

Today

Low-Rank Adaptation (LoRA)

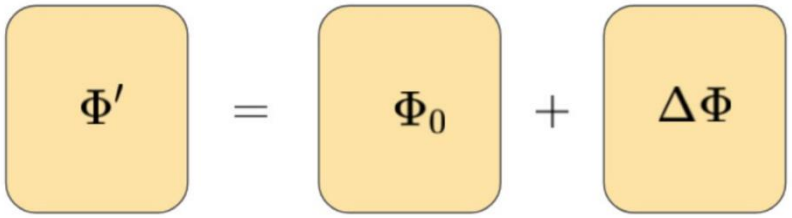
Tuning and Serving Many LoRAs

Fine-Tuning is Often Expensive

- **Full-parameter Fine-Tuning**: to accommodate new tasks/datasets
  - Updating all model parameters is resource-intensive

- **Parameter-Efficient Fine-Tuning (PEFT):** update a small subset of parameters
  - Tradeoff between efficiency and model quality

- **Low-rank Adaptation (LoRA)**
  - Pre-trained LLMs have a "low intrinsic dimension"
  - LLMs can still learn efficiently despite a low-dim reparametrization



Pre-trained
model weights

LoRA in One Slide

- **Insight**: insert small trainable weight matrix (as the side path) while keeping the base model frozen

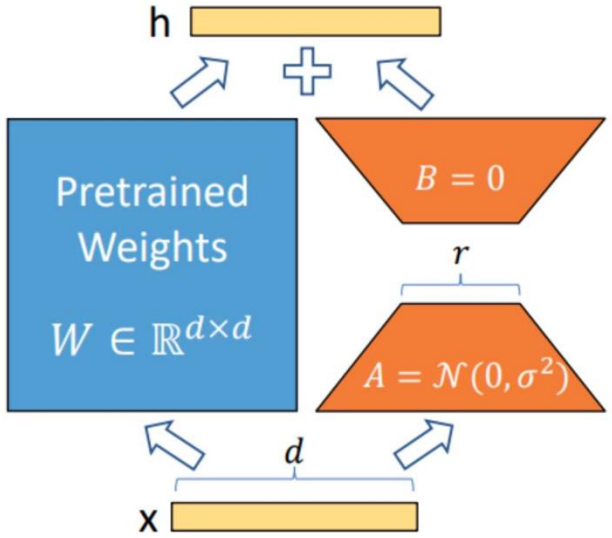


Figure 1: Our reparametrization. We only train A and B.

During training: for pre-trained weight $W_0 \in \mathbb{R}^{d \times k}$, W_0 is fixed

$$h = W_0 x + \Delta W x = W_0 x + BAx$$

$$B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d, k)$$

LoRA in One Slide

- **Insight**: insert small trainable weight matrix (as the side path) while keeping the base model frozen

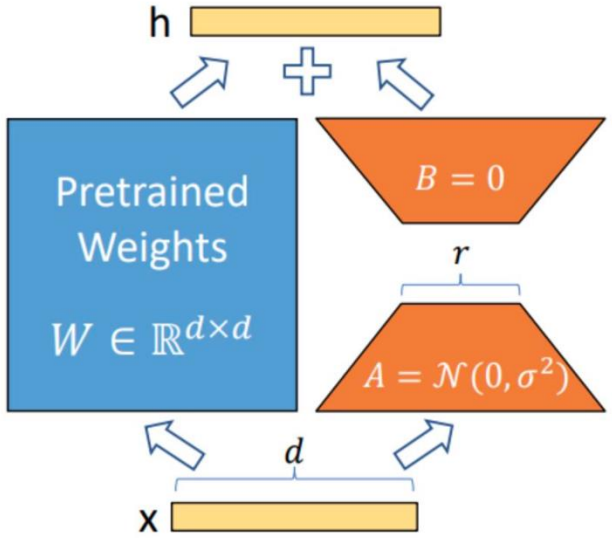


Figure 1: Our reparametrization. We only train A and B.

During training: for pre-trained weight $W_0 \in \mathbb{R}^{d \times k}$, W_0 is fixed

$$h = W_0 x + \Delta W x = W_0 x + BAx$$

$$B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d, k)$$

During inference:

$$W = W_0 + BA$$

Backward Computation of LoRA

- **Insight**: insert small trainable weight matrix (as the side path) while keeping the base model frozen

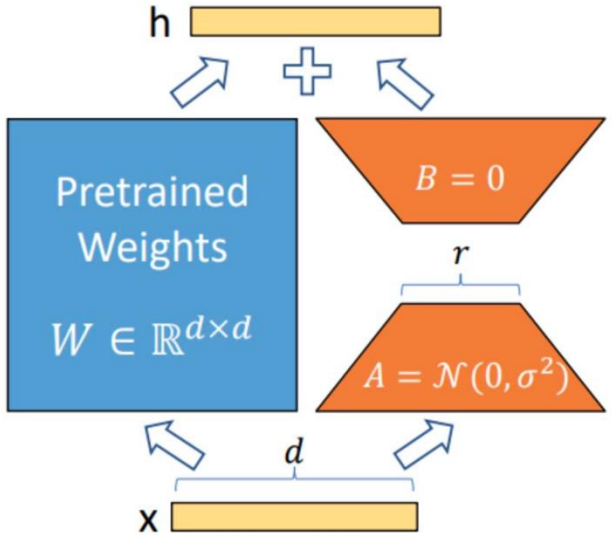


Figure 1: Our reparametrization. We only train A and B.

The gradients of the loss L with respect to A and B can be derived using the chain rule as follows:

$$\frac{\partial \mathcal{L}}{\partial A} = \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial h}{\partial A}$$

constant

$$= \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial (W_0 x + BAx)}{\partial A}$$

$$= \frac{\partial \mathcal{L}}{\partial h} \cdot Bx^T$$

W/o needing to compute the gradients for the base model (W_0)!

Backward Computation of LoRA

- **Insight**: insert small trainable weight matrix (as the side path) while keeping the base model frozen

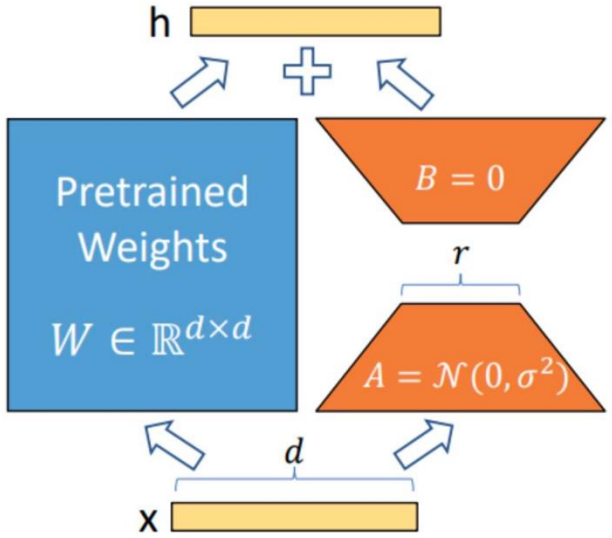


Figure 1: Our reparametrization. We only train A and B.

The gradients of the loss L with respect to A and B can be derived using the chain rule as follows:

$$\frac{\partial \mathcal{L}}{\partial A} = \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial h}{\partial A} \quad \text{(constant)}$$

$$= \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial (W_0 x + BAx)}{\partial A}$$

$$= \frac{\partial \mathcal{L}}{\partial h} \cdot Bx^T$$

$$\frac{\partial \mathcal{L}}{\partial B} = \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial h}{\partial B}$$

$$= \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial (W_0 x + BAx)}{\partial B}$$

$$= \frac{\partial \mathcal{L}}{\partial h} \cdot Ax^T$$

These partial derivatives are computed during the backward pass of backpropagation to update the parameters A and B accordingly.

Benefits of LoRA

Applying LoRA to Transformer

- In principle, LoRA can be applied to any weight matrices in DL
- LoRA's original paper only study changing the attention weights

$$W_q, W_k, W_v \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$$

Benefits of LoRA

Applying LoRA to Transformer

- In principle, LoRA can be applied to any weight matrices in DL
- LoRA's original paper only study changing the attention weights

$$W_q, W_k, W_v \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$$

Benefits

- No need to track optimizer state for frozen params, smaller checkpoint size (GPT-3: 1.2TB [before] → 350GB [training] → 35MB [inference])
- Negligible inference latency as r << d
- speed up during training

LoRA-based Fine Tuning Performs Well

	# of Trainable Parameters = 18M					
Weight Type	W_q	W_k	W_v	W_o	W_q, W_k	W_q, W_v
Rank r	8	8	8	8	4	4
WikiSQL (±0.3%)	70.4	70.0	73.0	73.2	71.4	**73.7**
MultiNLI (±0.1%)	91.0	90.8	91.0	**91.3**	**91.3**	**91.3**

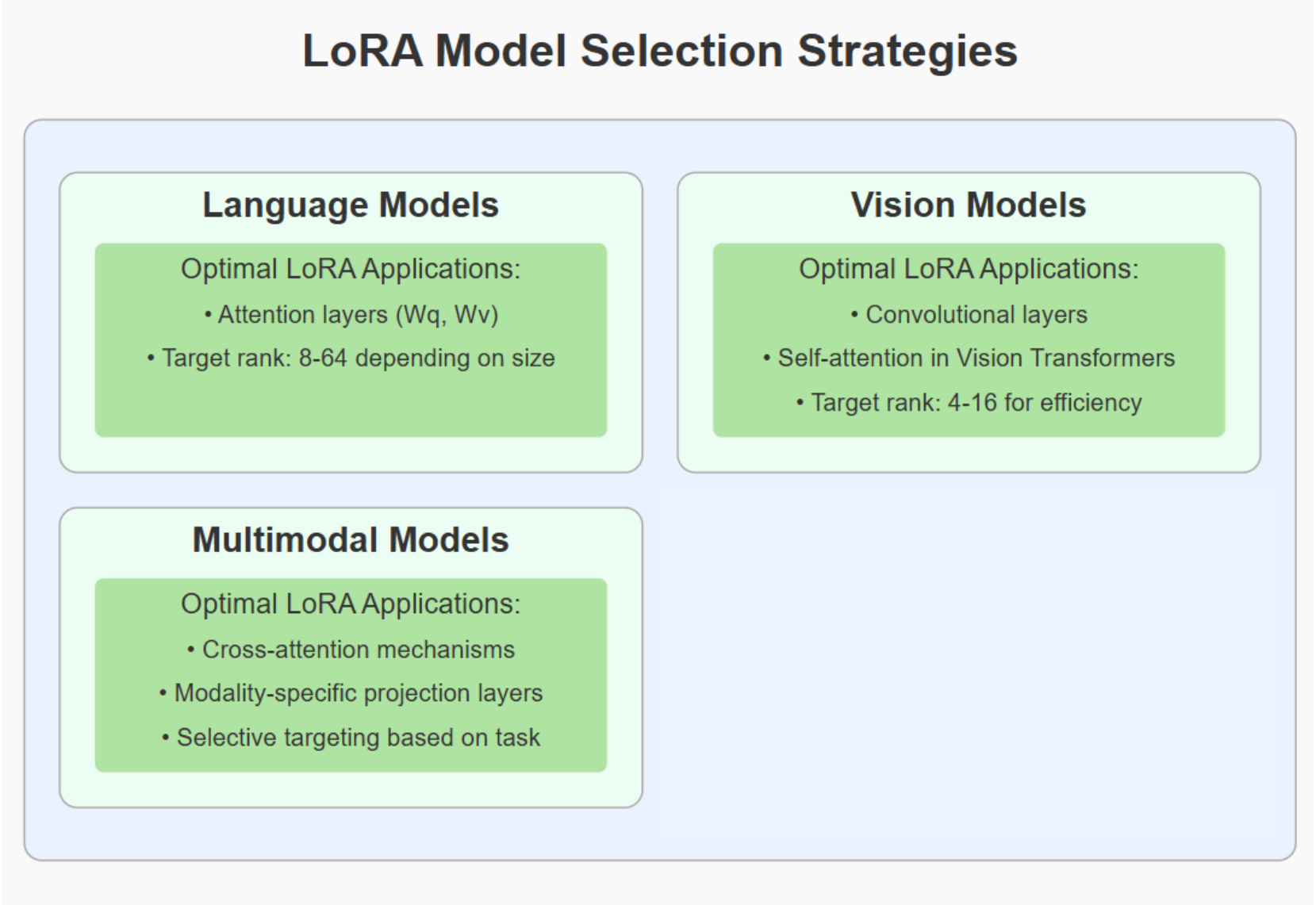
	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL(±0.3%)	W_q, W_v	73.4	73.3	**73.7**	**73.8**	73.5
	W_q	68.8	69.6	**70.5**	**70.4**	70.0
MultiNLI (±0.1%)	W_q, W_v	91.3	91.4	91.3	**91.7**	91.4

What is the optimal rank for LoRA?

"low intrinsic dimension"

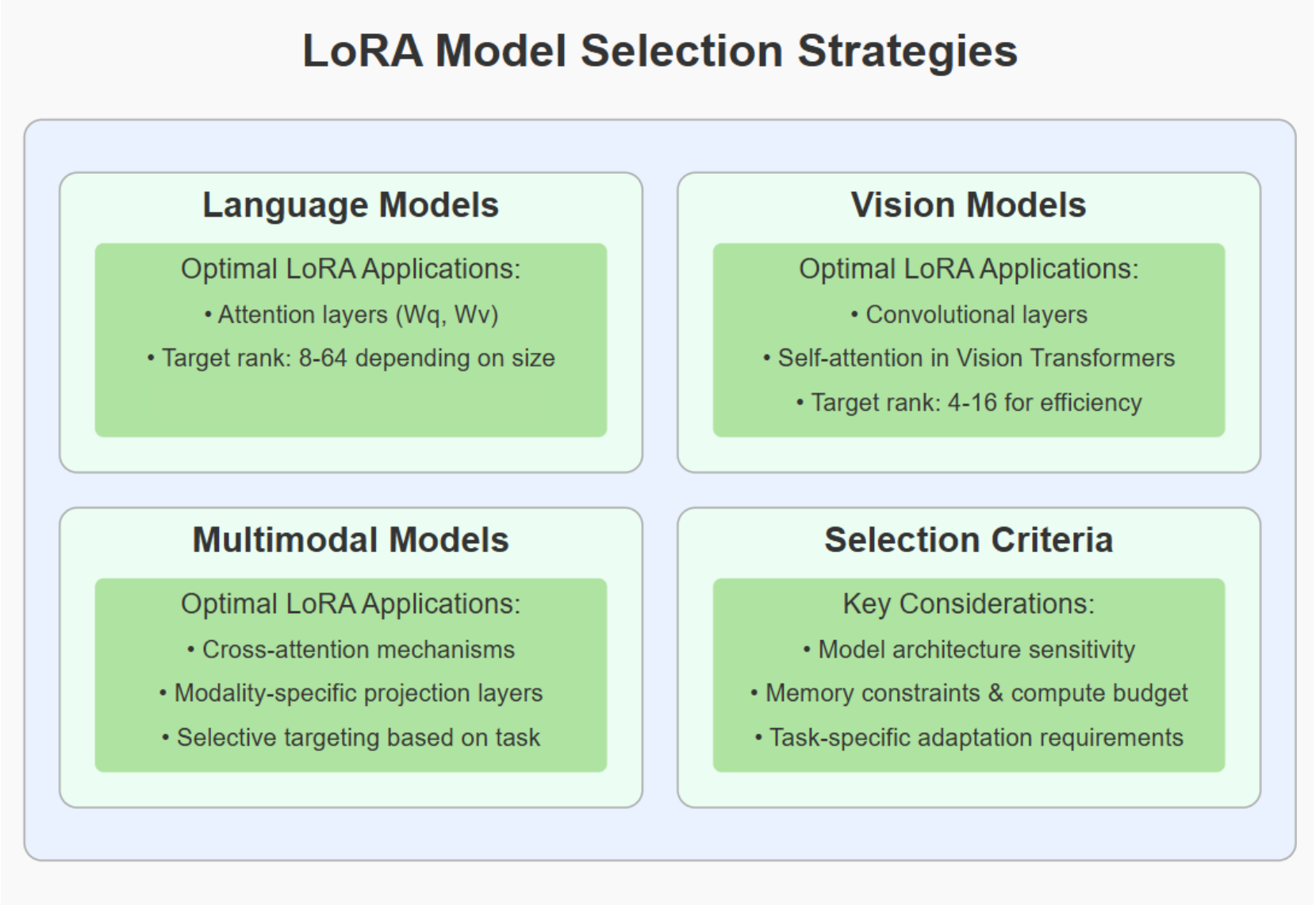
- increasing rank does not cover more meaningful subspaces, which suggests that a low-rank adaptation matrix is sufficient.

Design Choice of LoRA



LoRAs are mostly applied to the attention layer, why?

Design Choice of LoRA



LoRAs are mostly applied to the attention layer to capture token interactions

Today

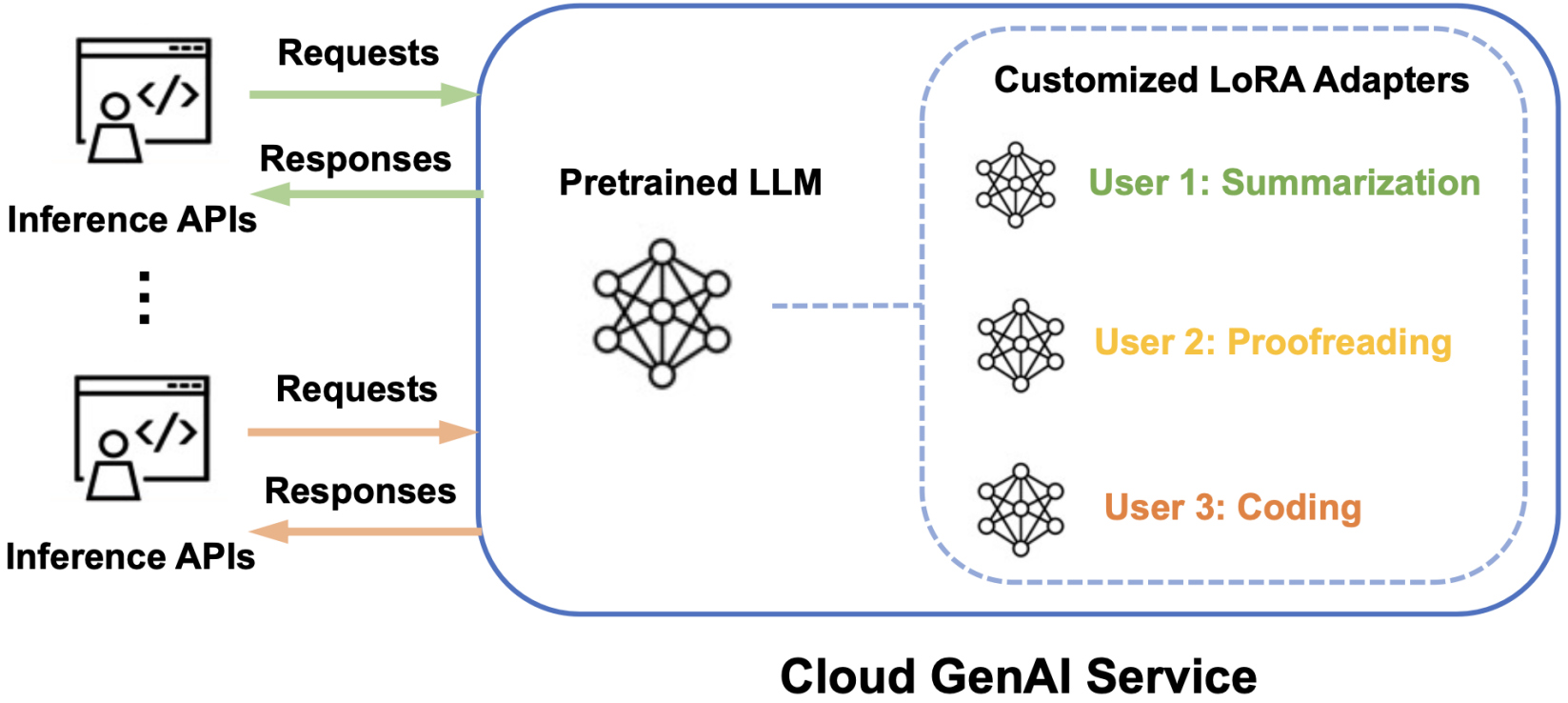
Low-Rank Adaptation (LoRA)

Tuning and **Serving Many LoRAs**

Pannel Discussion

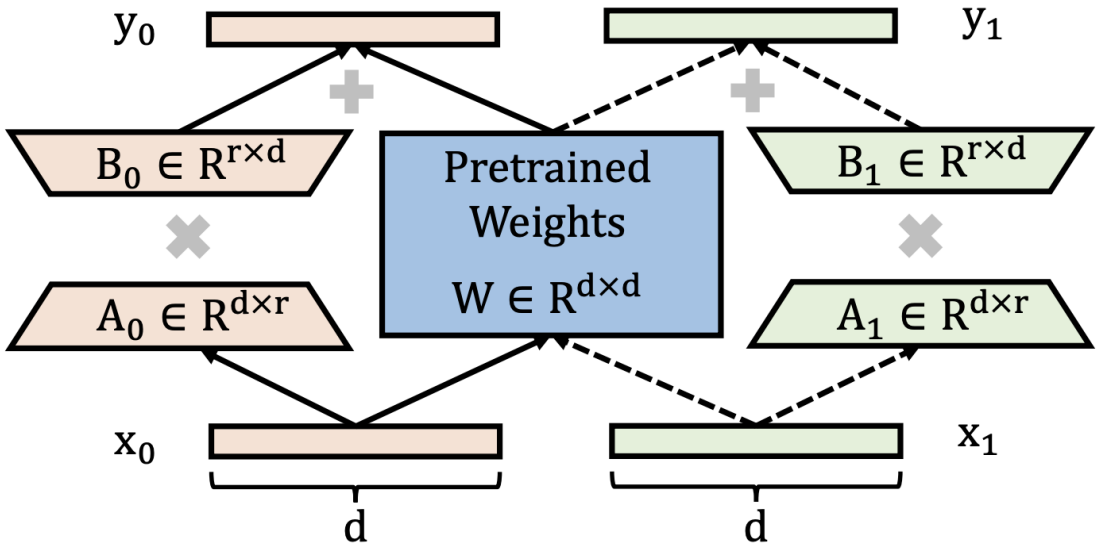
From Fine-Tuning to Grounding LoRAs

- **Cloud providers may host many adapters for an LLM**
  - Different users may use different adapters for different scenarios



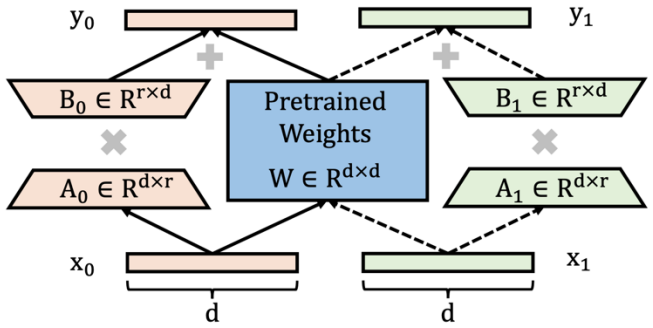
From Fine-Tuning to Grounding LoRAs

- **Setting up model replicas for each user (LoRA) under-utilizes GPUs**
  - We can co-locate multiple LoRAs while sharing one base model



Main Idea of S-LoRA

- **Store adapter weights in main memory (e.g., DRAM)**
  - GPU memory too small to store there



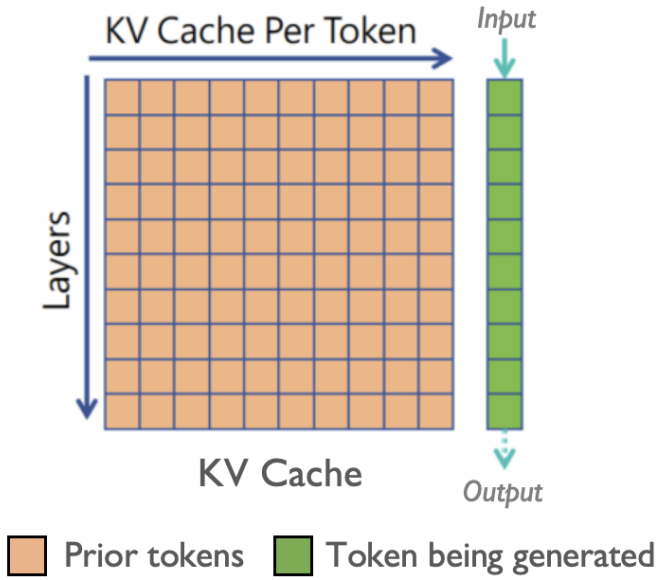
- **Issues:**
  - Dynamic fetching leads to memory fragmentation and I/O overhead
  - Hard to batch computation of adapters in non-contiguous memory

S-LoRA: Serving Thousand of Concurrent LoRA Adapters (MLSys'24)

Memory Management in S-LoRA

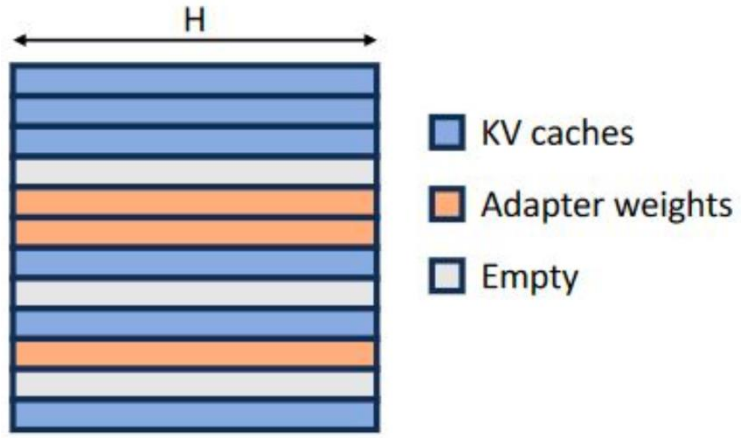
- **Loading and unloading of active adapters**
  - Inactive adapters -> Main memory
  - Active adapters -> GPU memory

- **Naive loading and unloading problematic**
  - Causes memory fragmentation
  - Latency overhead in switching data



Memory Management in S-LoRA

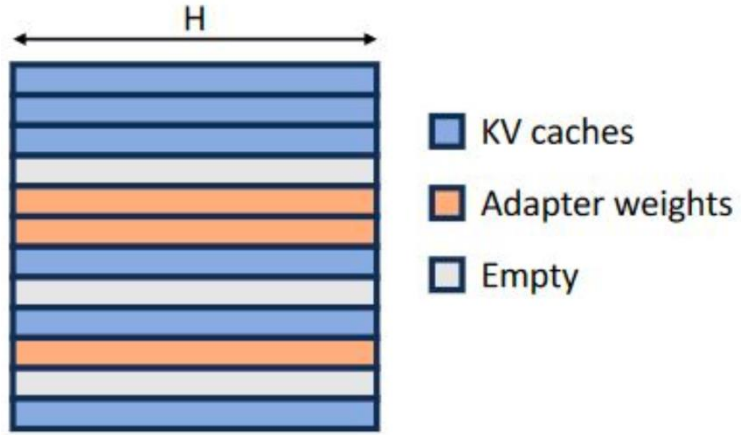
- **Loading and unloading of active adapters**
  - Inactive adapters -> Main memory
  - Active adapters -> GPU memory
- **Naive loading and unloading problematic**
  - Causes memory fragmentation
  - Latency overhead in switching data



Memory Management in S-LoRA

- **Loading and unloading of active adapters**
  - Inactive adapters -> Main memory
  - Active adapters -> GPU memory

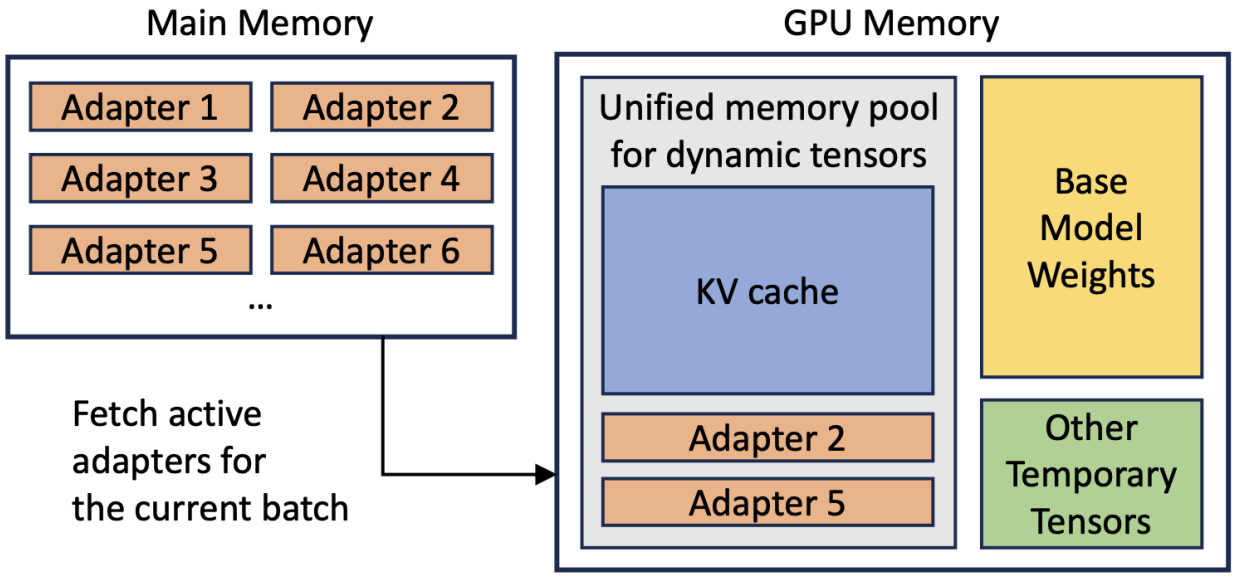
- **Naive loading and unloading problematic**
  - Causes memory fragmentation
  - Latency overhead in switching data

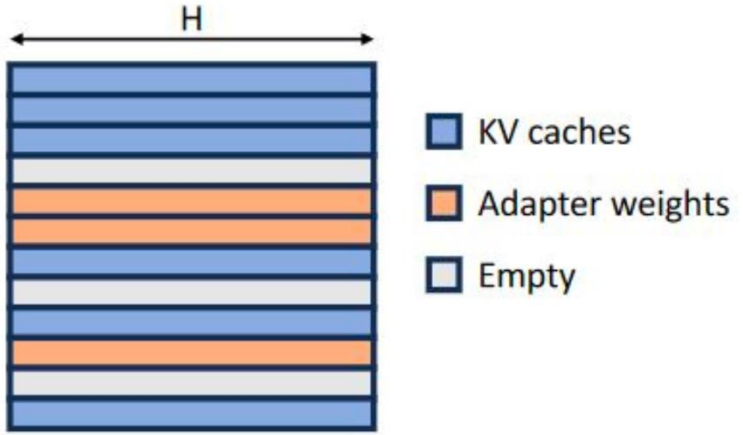


- **Unified paging: KV cache and adapter weights share same memory pool**
  - Both KV cache and adapter weights allocated on request input and deallocated on request completion
  - Interleaved storage to reduce fragmentation

Memory Management in S-LoRA

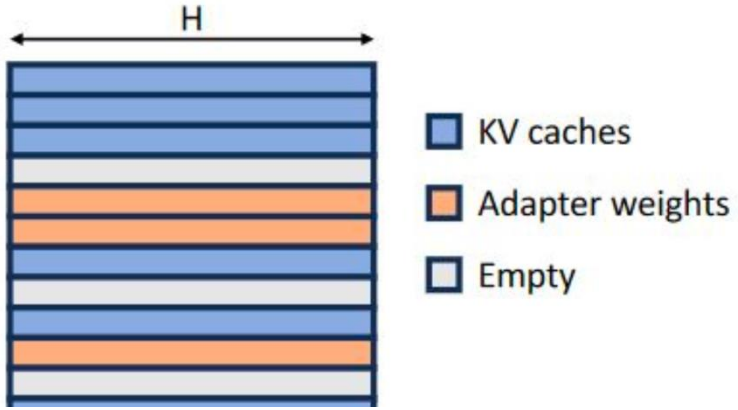
- **Prefetching and overlapping**
  - Predict adapter weights needed for next batch when decoding current batch





Memory Management in S-LoRA

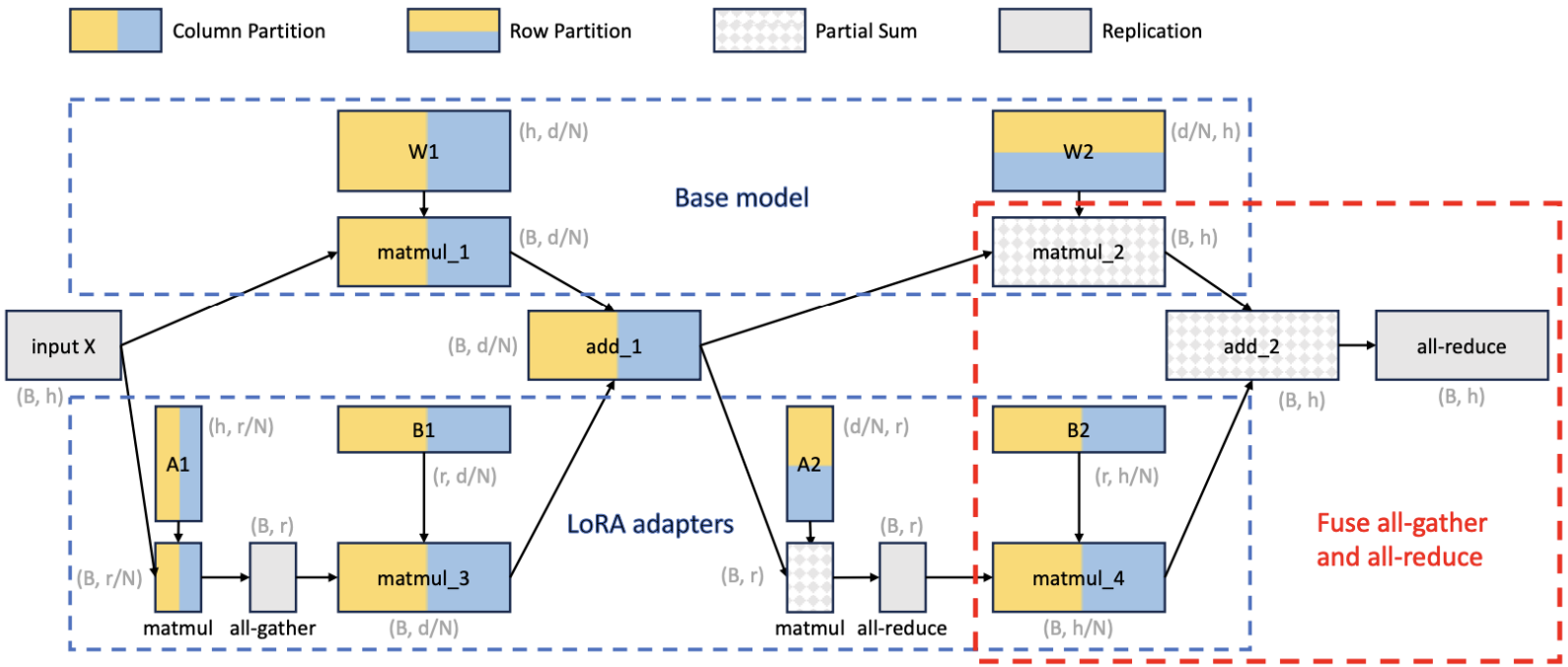
- **Prefetching and overlapping**
  - Predict adapter weights needed for next batch when decoding current batch



- **S-LoRA = Scalable LoRA**
  - Unified Paging: Unified memory pool to reduce fragmentation, increase batch size
  - Heterogenous Batching: Batching to allow concurrency of different adapters
  - S-LoRA Tensor Parallelism: Split up adapter weights for inference across GPUs

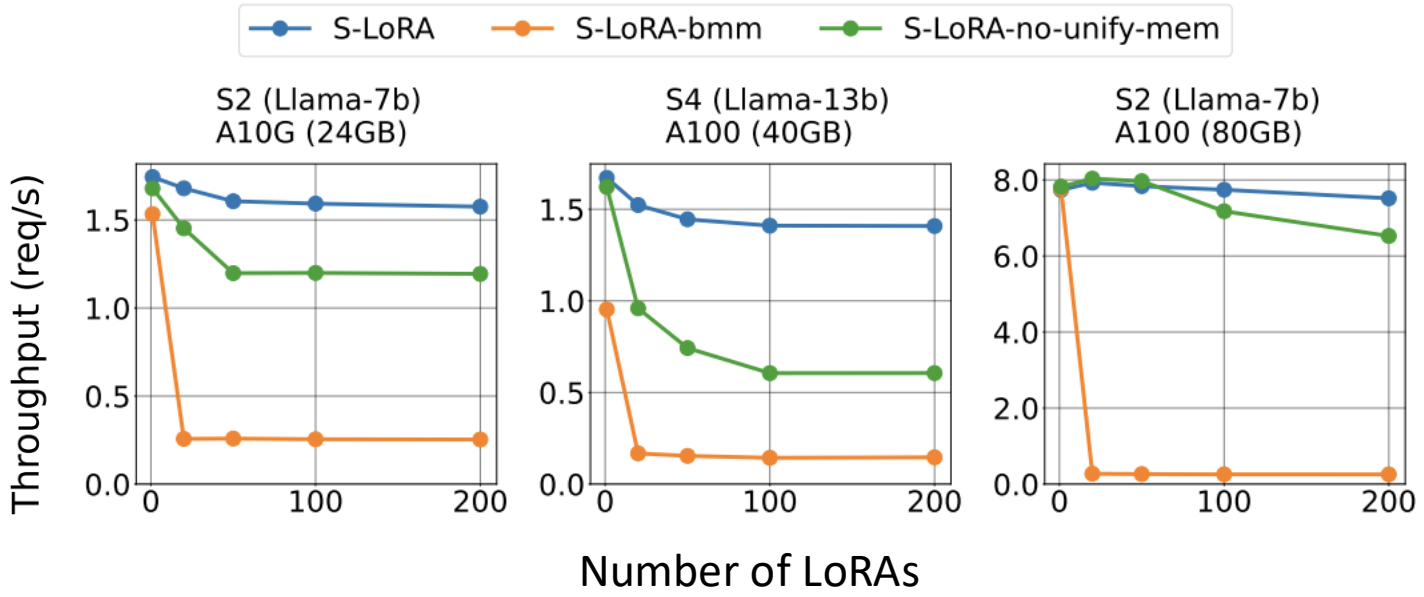
Model Parallelism Supports S-LoRA

- **Models may be too large to fit into a single machine → model parallelism!**
  - Act very similar to single-model parallelism as LoRA r << d



S-LoRA Outperforms Naïve Serving without Co-locating LoRAs

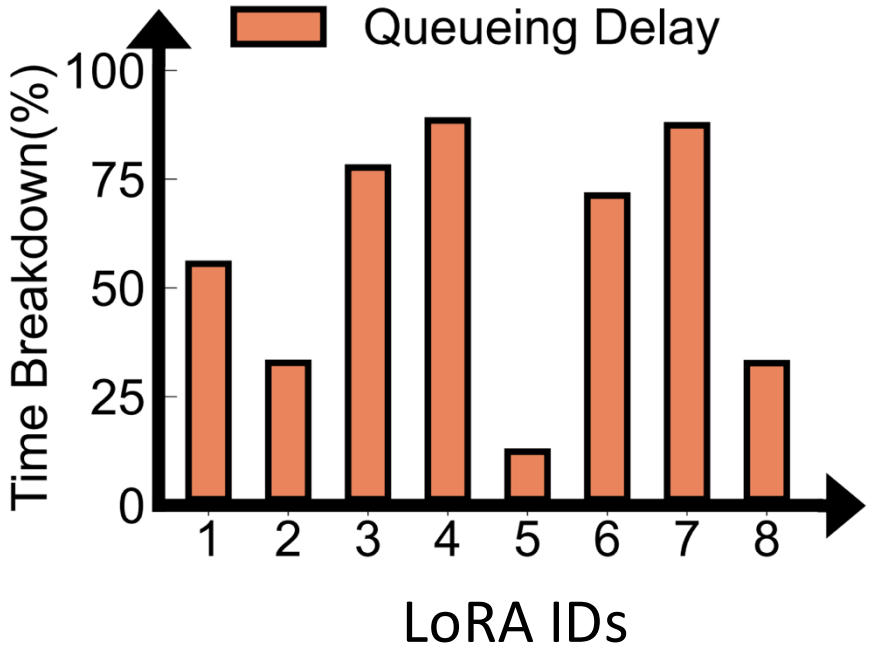
- **S-LORA-bmm:** S-LoRA without unified memory management and customized kernels
- **S-LORA-no-unify-mem:** S-LoRA without unified memory management



From S-LoRA to dLoRA[OSDI'24]

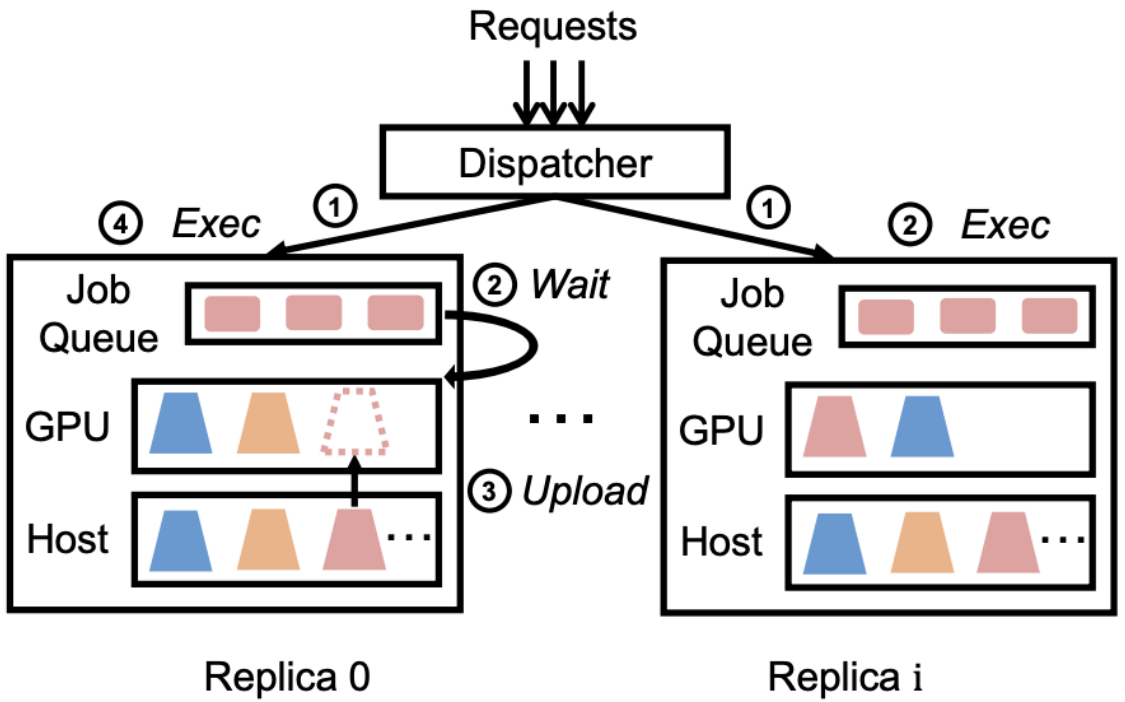
- **S-LoRA assumes all-or-nothing serving**
  - Use that LoRA or not

The burst of variable requests leads to severe load imbalance under static LoRA placement



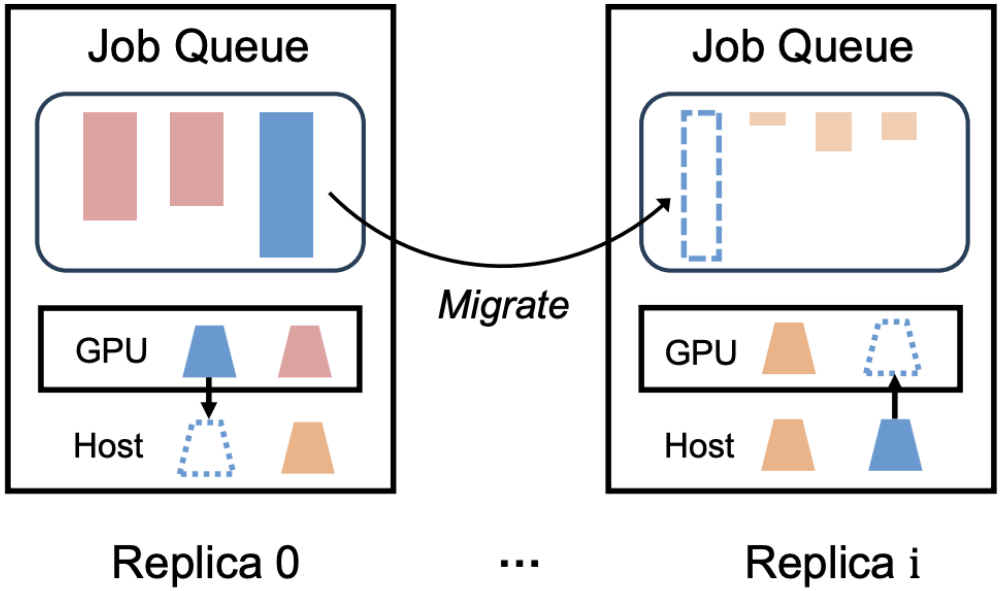
dLoRA: Dynamically Orchestrating Requests and LoRAs

- **Key Insights**
  - Set up multiple model replicas and replicate hot LoRAs
  - Considering both adapter loading time and queuing delay to balance the load



dLoRA: Dynamically Orchestrating Requests and LoRAs

- **Key Insights**
  - Set up multiple model replicas and replicate hot LoRAs
  - Considering both adapter loading time and queuing delay to balance the load
  - Reactively migrate workload to balance the load



Summary: Fine-Tuning Techniques

- **Parameter-Efficient Fine-Tuning (PEFT):** update a small subset of parameters
  - Low-Rank Adaptation (LoRA): insert small trainable weight matrix
  - Lower memory footprint and fine-tuning computation with little quality drop

- **Serving Many LoRAs:** practical deployments often host many LoRAs
  - S-LoRA: multiple LoRAs share the base model, offloading cold LoRAs to DRAM
  - dLoRA: S-LoRA + (reactive) load balance + (proactive) migration

Open problems?

(e.g., how to cache LoRA and KVCache)